

# Git Version Control

Siddharth Prasad

*Department of Computer Science Engineering*

*MIT Manipal, India*

siddharthprasad61@gmail.com

**Abstract**—Version control systems are fundamental to modern software engineering, enabling distributed teams to collaborate efficiently and maintain comprehensive project histories. This report presents a detailed analysis of Git, the industry-standard distributed version control system. It outlines the core architecture of Git repositories and provides a systematic examination of essential version control commands. Fundamental operations, including staging, committing, fetching, and pulling, are discussed to establish a baseline of local and remote repository management. Furthermore, the report explores advanced history manipulation and branching techniques—such as rebasing, cherry-picking, and amending commits—which are critical for resolving conflicts and maintaining a clean project history. Ultimately, this comprehensive guide serves as a practical reference for understanding Git’s underlying mechanics, equipping developers with the necessary commands to streamline collaboration and safeguard code integrity in complex software projects.

## I. INTRODUCTION

A Version Control System (VCS) is a fundamental software engineering tool designed to record modifications to a file or a set of files over time, enabling developers to recall specific versions on demand. The primary objective of a VCS is to facilitate concurrent development among multiple contributors, prevent the accidental overwriting of code, and maintain a rigorous audit trail of all project modifications.

The architecture of version control has evolved through three primary paradigms: Local, Centralized, and Distributed systems.

### A. What is Git?

B. What Is Git Git is a distributed version control system designed to handle everything from small to large projects with speed and efficiency. It allows each developer to have a full copy of the repository, including the history. Key features include branching, merging, data integrity via SHA-1 hashing, and the ability to work offline.

### B. Local and Centralized Version Control

Early version control relied on Local VCS (LVCS), which simply maintained patches (differences between files) in a local database on a single computer. While this prevented basic file-overwriting errors, it did not facilitate team collaboration.

To address collaborative needs, Centralized Version Control Systems (CVCS), such as Subversion (SVN) and Perforce, were developed. A CVCS utilizes a single central server that stores all versioned files, with clients checking out specific

files to their local machines. While this centralized approach allowed developers to see what others were doing and gave administrators granular control over permissions, it introduced a critical vulnerability: a single point of failure. If the central server experienced downtime, collaboration ceased entirely; if the central database corrupted without a backup, the entire project history was permanently lost.

### C. Distributed Version Control Systems (DVCS)

To mitigate the inherent risks of centralized servers, the Distributed Version Control System (DVCS) model was introduced. In a DVCS framework, clients do not merely check out the latest snapshot of the files; rather, they fully mirror the entire repository, including its complete chronological history.

Consequently, if a central server fails, any client’s local repository can be utilized to completely restore the server. This decentralized architecture inherently provides multiple redundant backups of the project data. Furthermore, a DVCS allows developers to execute the vast majority of version control operations—such as committing, branching, and inspecting history—entirely offline, without suffering from network latency. This robust, fault-tolerant architecture forms the underlying foundation of Git.

## II. SETUP AND CONFIG

- `git config --global user.name "Your Name"` (Set global username)
- `git config --global user.email "you@example.com"` (Set global email)
- `git config --list` (List all config)
- `git config --global --list` (List global config)
- `git config --local --list` (List local config)

## III. WORKFLOW

- `git status` (Show working tree status)
- `git add <file >` (Stage a specific file)
- `git add` (Stage all changes)
- `git commit -m "message"` (Commit staged changes)
- `git commit -am "message"` (Stage all and commit)
- `git log` (Show commit history)
- `git log --oneline --graph --all` (Compact commit history graph)
- `git diff` (Show unstaged changes)
- `git diff --staged` (Show staged changes)
- `git show <commit >` (Show details of a commit)

## IV. BRANCHING AND MERGING

### A. Branching

Branching in Git refers to the mechanism of diverging from the primary line of development to work independently without affecting the stable codebase. In contrast, Git's branching model is highly lightweight. A branch in Git is essentially a movable pointer directed toward a specific commit snapshot.

### B. Merging

Merging is the fundamental process of integrating independent, divergent lines of development back into a single, unified history. Once work within an isolated feature branch is completed and validated, the git merge command is utilized to incorporate those changes into a target branch (typically the main branch).

### C. Commands

- git branch (List branches)
- git branch -r (List remote branches)
- git branch -a (List all branches)
- git branch branch-name (Create a new branch)
- git checkout <branch-name> (Switch to a branch)
- git checkout -b <branch-name> (Create and switch to a branch)
- git merge <branch-name> (Merge branch into current)
- git branch -d <branch> (Delete a branch (safe))
- git branch -D <branch> (Force delete a branch)

## V. STASH

### A. Stashing

The git stash command is a mechanism used to temporarily shelve uncommitted local changes, returning the working directory to a clean state. This allows developers to seamlessly switch branches or pull remote updates without being forced to make a premature commit or risk losing incomplete work. The stashed modifications are saved on a stack and can be reapplied at any time.

- git stash (Stash local changes)
- git stash push -m "message" (Stash with a message)
- git stash list (List stashes)
- git stash show (Show stash changes)
- git stash pop (Apply and remove latest stash)
- git stash apply (Apply latest stash (keep it))
- git stash drop (Remove latest stash)
- git stash clear (Remove all stashes)

## VI. THE COMMIT PROCESS

Git's commit mechanism involves:

1. Staging with git add.
2. Committing with git commit -m 'message'.
3. Viewing history via git log. git log shows multiple commits, while git show displays details (diffs, author, message) of a specific commit

## VII. REBASING

Rebasing is an alternative to merging that integrates changes by moving an entire branch to a new starting point. It works by copying the commits from a feature branch and reapplying them directly on top of another branch. This rewrites the project history to create a clean, linear progression, eliminating the complex, web-like structures created by standard merge commits.

### A. Commands

- git rebase main ( Moves commits from the current branch onto another branch )
- git rebase -onto target-branch base-branch ( Replays commits onto a specific commit or branch, excluding the divergence point.)
- git pull --rebase ( Fetches remote updates and automatically rebases local commits on top of them.)
- git rebase -i HEAD~3 (Opens an editor to modify, reorder, squash, or drop commits.)
- git rebase --continue (Stages resolved files and resumes the rebase process.)
- git rebase --abort ( Cancels the rebase and restores the branch to its pre-rebase state.)
- git rebase --skip (Skips the current conflicted commit and continues.)

## VIII. TAGGING

Tagging in Git is a mechanism used to permanently mark specific, significant points in a repository's history. While branch pointers move forward as new commits are added, a tag remains statically attached to a single commit. It acts as an immutable bookmark, most commonly used to denote software release versions.

### A. Commands

- git tag (List tags)
- git tag <tagname> (Create a lightweight tag)
- git tag -a <tagname> -m "msg" (Create an annotated tag)
- git show <tagname> (Show tag details)
- git push origin <tagname> (Push a tag)
- git push origin --tags (Push all tags)

## IX. UNDOING, RECOVERY, AND LOGS

### A. Git log and Reflog

git log displays a project's commit history chronologically. 'git reflog' tracks HEAD movements, even those not visible in normal logs, including resets and checkouts

### B. Undoing and Resetting

1) *Undoing*: In Git, undoing modifications allows developers to safely reverse errors without corrupting the shared project history. For previously committed changes, the git revert command is the standard practice. Rather than deleting the erroneous commit, git revert generates a brand new commit that precisely inverts the changes, thereby preserving a linear, auditable history. For uncommitted local modifications, git

restore can be utilized to discard changes and return files to their last tracked state.

2) *Resetting*: The git reset command provides a powerful mechanism to forcefully rewind the current branch pointer back to a specific historical commit. Unlike reverting, resetting physically alters and removes existing commit history. It operates in three primary modes: soft (preserves the discarded commits as staged changes), mixed (keeps the changes in the working directory but unstages them), and hard (permanently deletes all modifications after the target commit). Because it rewrites history, git reset is typically restricted to local branches that have not yet been shared with a team.

### C. Commands

- git restore services <file name>—for restoring the entire directory
- git restore --staged <file name>—for undoing the changes that were done using the git add command
- git restore . — for undo all the changes in the entire repository
- git reset --hard — for retrieving the files which had been manually been deleted
- git reset— used for unstaging the files
- git reset --soft <commit>— Moves HEAD but keeps staged changes.
- git revert — for fixing an old mistake without erasing it that is commit

## X. PUSH AND PULL

### A. Push

The git push command transmits committed local modifications to a remote repository. This operation updates the remote branches, synchronizing the shared central server with the developer's local environment and making the newly created commits accessible to the rest of the development team

### B. Pull

The git pull command retrieves the latest modifications from a remote repository and immediately integrates them into the current local branch. It fundamentally operates as a combination of git fetch followed by git merge, ensuring the developer's local workspace is fully synchronized with the shared project state.

### C. Commands for Push

- git push origin <branch-name>—To push a specific branch to a specific remote
- git push --all —Pushes all local branches to the remote
- git push --tags — Pushes all local tags to the remote
- git push --force — Forces a non-fast-forward update, overwriting remote history
- git push --delete origin <branch-name>—Deletes a branch from the remote repository.

### D. Commands for Pull

- git pull—Pulls the latest changes from the tracked remote branch into the current local branch.
- git pull <remote>— Pulls from a specific remote but uses the configured upstream branch
- git pull <remote><branch>—Pulls a specific branch from a specific remote
- git pull --all—Fetches and pulls from all configured remote repositories
- git pull --rebase — Fetches changes and rebases local commits on top of the remote changes, maintaining a linear history without merge commits.
- git pull --no-rebase — Forces a merge operation, overriding any global rebase configuration
- git pull --tags—Fetches and pulls tags along with the branch changes

## XI. CHERRY-PICK

It's a very straightforward way of saying that you would like to copy a series of commits below your current location (HEAD).

## XII. GIT DESCRIBE

Since tags serve as such great "anchors" in the codebase, git has a command to describe where you are relative to the closest "anchor" (aka tag). And that command is called git describe

## XIII. FETCH

git fetch performs two main steps: It downloads the commits that the remote has but are missing from our local repository. It updates where our remote branches point.

### A. Commands

git fetch—Fetches from the default remote  
git fetch <remote>—Fetches from a specific remote  
git fetch <remote><branch>—Fetches a specific branch  
git fetch --all —Fetches from all configured remotes simultaneously

## XIV. MERGE CONFLICT

A merge conflict occurs when Git is unable to automatically reconcile differences between two divergent branches during a merge or rebase operation. This typically happens when multiple developers modify the exact same lines of code within the same file, or when one developer edits a file while another deletes it. Rather than arbitrarily overwriting data or guessing which version is correct, Git halts the integration process and safely leaves the repository in a conflicted state. To highlight the discrepancies, Git injects standard conflict markers (such as <<<<<<<, =====, and >>>>>>>) directly into the affected files. To resolve the issue, the developer must manually open these files, evaluate the competing changes, and edit the code to the desired final state. Once the conflict markers are removed and the code is reconciled, the resolution is finalized by staging the file using the git add command and recording the integration with git commit.

## XV. COMMON GIT COMMANDS

The following is a summary of the fundamental Git commands utilized for local and remote repository management:

- `git status` – Shows branch and file states.
- `git add` – Stages changes for commit.
- `git commit` – Records staged changes permanently.
- `git log` – Lists commit history.
- `git show` – Displays detailed info about a commit.
- `git switch` – Switches branches safely.
- `git checkout` – Switches branches or restores files.
- `git fetch` – Downloads commits without merging.
- `git pull` – Fetches and merges remote changes.
- `git push` – Uploads commits to remote repository.
- `git restore` – Restores modified files.
- `git reset` – Reverts or unstages commits.
- `git rm` – Removes files from repository.
- `git cherry-pick <Commit1><Commit2><...>`—Copies a single, specific commit and applies it to the current branch.
- `git branch -f main HEAD <number of times to move the commits>`—Forcefully moves the main branch pointer backward by a specified number of commits.
- `git commit -amend` —Updates the most recent commit to change its message or include newly staged files.
- `git fakeTeamwork <number of times>`—Simulates remote activity by adding a specified number of commits to the remote branch.
- `git pull -rebase` — is shorthand for a fetch and a rebase
- `git checkout HEAD^` — for moving to the upward in a commit tree

## XVI. COMMON ISSUES AND FIXES

- Detached HEAD – Switch back using `git switch <branch>`
- Merge conflicts – Resolve manually or with GUI tools
- Non-fast-forward errors – Pull latest changes and merge
- Lost commits – Recover using `git reflog` and `git checkout <hash>`.

## XVII. CONCLUSION

Git provides a resilient and efficient system for managing source code and collaboration. By understanding commits, branching, merging, rebasing, and recovery mechanisms, developers can maintain project integrity and efficiency. Mastery of Git ensures better project control, coordination, and traceability in professional environments.

## XVIII. REFERENCES

### REFERENCES

- [1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York, NY, USA: Apress, 2014.
- [2] D. Spinellis, "Git," *IEEE Software*, vol. 29, no. 3, pp. 100-101, May-June 2012.
- [3] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, "The promises and perils of mining Git," in *Proc. 6th IEEE Int. Working Conf. Mining Software Repositories*, 2009, pp. 1-10.
- [4] Git Development Community, "Git Reference Manual," 2026. [Online]. Available: <https://git-scm.com/docs>.

- [5] P. Cottle, "Learn Git Branching," 2026. [Online]. Available: <https://learngitbranching.js.org>.